

山から粒子が落ちる

DirectX 11 SPH 流体シミュレーション

技術解説書

プロジェクト: water_from_mountain

リポジトリ: https://github.com/githubtarohario/water_from_mountain

対象バージョン: コミット 28a8593 (2026-09-13)

参考: 図6.37 表面生成.mp4

目次

1. 概要と成果物
2. システム構成
3. 開発環境とビルド
4. 地形生成（手続き的高さマップ）
5. 岩テクスチャ（WIC 読み込みと手続き生成）
6. OBJ 地形の取り込み（Blender 連携）
7. SPH 流体シミュレーション
8. 近傍探索（空間ハッシュ + カウンティングソート）
9. 境界処理と安定化
10. 粒子のレンダリング（スフィア・インポスター）
11. 表面生成（スクリーンスペース流体レンダリング）
12. 描画パイプラインと定数バッファ
13. カメラ・入力・メインループ
14. 性能評価
15. 調整パラメーター一覧
16. 既知の制約と拡張案
17. 付録: ファイル一覧・参考文献

1. 概要と成果物

1.1 目的

参考動画「図6.37 表面生成.mp4」に示される、岩肌の山に刻まれた蛇行する谷を流体粒子が流れ落ち、粒子群から水面（表面）を生成して描画するシミュレーションを、DirectX 11 と C++ で再現する。前半は速度で着色した粒子として、後半は「表面生成」により連続した水面として表示する。

1.2 成果物

実行環境	Windows 10/11、DirectX 11 対応 GPU、Visual Studio 2019/2022
言語 / API	C++17、Direct3D 11、HLSL Shader Model 5.0、DirectXMath、WIC、OpenMP
外部ライブラリ	なし（Windows SDK のみ）
物理モデル	WCSPH（弱圧縮性 SPH、Müller et al. 2003）、CPU 並列（OpenMP）
表面生成	スクリーンスペース流体レンダリング（van der Laan et al. 2009 の簡略版）
規模	粒子 12,000 個、地形 257×257 頂点（OBJ 取り込み可）、ソース約 4,300 行
性能	粒子モード 60～120 fps、表面モード 55～80 fps（8 コア CPU + ミドルレンジ GPU）

1.3 実行結果

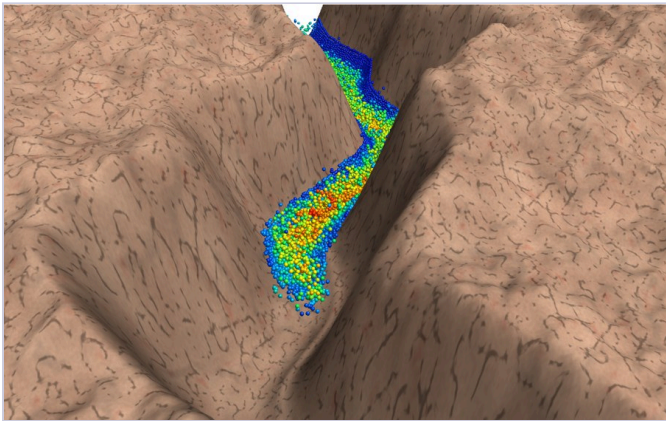


図 1-1 粒子モード（速度で青→緑→赤に着色）

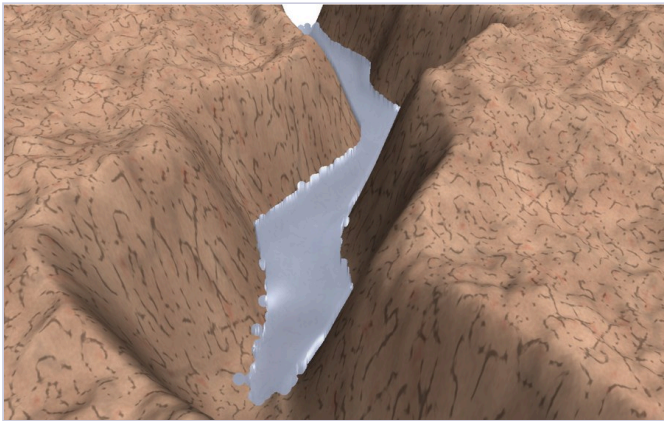


図 1-2 表面生成モード（水面として描画）

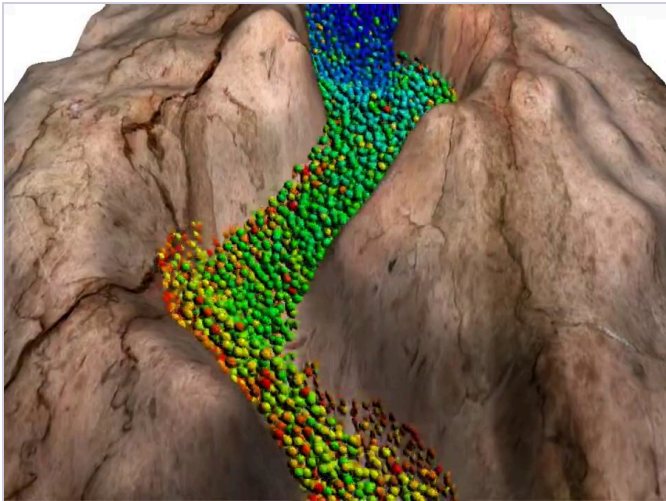


図 1-3 参考動画（粒子表示）

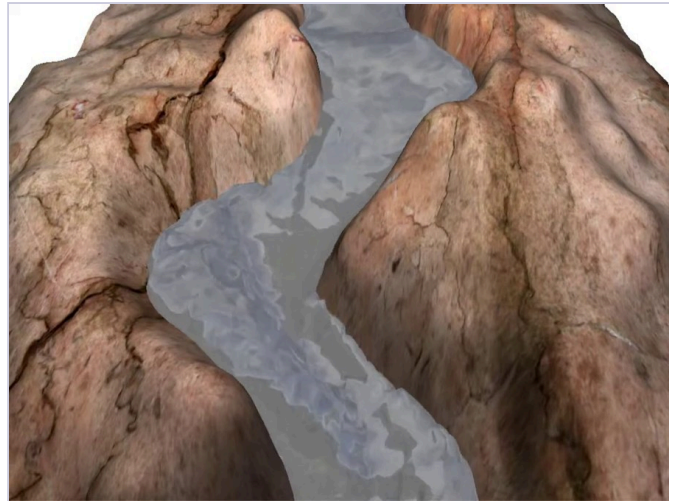


図 1-4 参考動画（表面生成）

1.4 操作

入力	動作	入力	動作
マウス左ドラッグ	カメラ回転	1 / 2	粒子モード / 表面生成モード
マウス右ドラッグ	カメラ平行移動	Space	一時停止 / 再開
ホイール	ズーム	R / Esc	粒子リセット / 終了

2. システム構成

2.1 モジュール

モジュール	ファイル	責務
アプリケーション	main.cpp	ウィンドウ、入力、フレームループ、定数バッファ更新、モジュールの結線
Graphics	Graphics.h/.cpp	D3D11 デバイス・スワップチェーン・RTV/DSV・共通ステート、シェーダーコンパイル、バッファ更新
FrameConstants	FrameConstants.h	全シェーダー共通の定数バッファ構造体 (b0)
Camera	Camera.h/.cpp	オービットカメラ (ビュー / 射影行列)
Noise	Noise.h/.cpp	パーリンノイズ、fBm、尾根状ノイズ (周期版を含む)
Terrain	Terrain.h/.cpp	OBJ 読み込み or 高さマップ生成、メッシュ、岩テクスチャ、衝突用の高さ / 法線問い合わせ
TextureLoader	TextureLoader.h/.cpp	WIC による画像読み込み、ミップマップ付きテクスチャ作成
MeshLoader	MeshLoader.h/.cpp	Wavefront OBJ の読み込み (多角形の三角形化、法線計算)
SPH	SPH.h/.cpp	流体シミュレーション本体 (近傍探索、密度・圧力・力、積分、境界、エミッタ)
FluidRenderer	FluidRenderer.h/.cpp	粒子の球描画、スクリーンスペース流体 (深度 → ぼかし → 合成)
シェーダー	shaders/*.hlsl(i)	Terrain / Particle / FluidSurface / Common

2.2 データフロー

[起動時]
assets/terrain.obj → MeshLoader → 座標変換/自動フィット → 頂点/インデックスバッファ (GPU) └─┬─ BakeHeightMap → 高さマップ (CPU, float 257x257) └─(無ければ) Noise → Terrain::BuildHeightMap → 高さマップ └─ BuildMesh → 頂点/インデックスバッファ (GPU, IMMUTABLE)
assets/rock.png → WIC → TextureLoader → 岩テクスチャ (GPU, R8G8B8A8 + mips) └─(無ければ) Noise → CreateProceduralRockTexture ─┘
[毎フレーム]
入力 → Camera → FrameConstants (View/Proj/光源) → cbuffer b0
SPH::Update(dt) ... 高さマップを参照して衝突 └─ GetRenderData → float4(x,y,z,速さ) → Map → StructuredBuffer (GPU)
Terrain::Render → バックバッファ + 深度
FluidRenderer::Render └─ 粒子モード: 球インポスター → バックバッファ └─ 表面モード: 球の深度 → depthTex → ぼかし×N → 法線復元・陰影 → 合成 → バックバッファ

2.3 設計方針

- 外部依存ゼロ: DirectXTK や stb_image を使わず、WIC と D3DCompiler など Windows SDK 標準のみで構成。

- **実行時シェーダーコンパイル:** HLSL は `D3DCompileFromFile` で起動時にコンパイル。シェーダーの修正に再ビルドが不要。
- **CPU 物理 + GPU 描画:** SPH は CPU（OpenMP）で計算し、位置と速さのみを毎フレーム GPU へ転送（ $12,000 \times 16 \text{ B} = 192 \text{ KB}$ ）。
- **頂点バッファ不要の描画:** 粒子は `SV_VertexID` / `SV_InstanceID` からビルボードを生成し、フルスクリーンパスも 3 頂点の三角形を生成する。
- **フォールバック:** 画像がなければノイズ生成、デバッグレイヤーがなければ通常デバイス、というように環境差で落ちない構成。

3. 開発環境とビルド

コンパイラ	MSVC v143 (Visual Studio 2022) 、 <code>/std:c++17 /O2 /openmp /utf-8</code>
プリプロセッサ定義	<code>NOMINMAX</code> (windows.h の min/max マクロが <code>std::min/max</code> と衝突するのを防 止) 、 <code>UNICODE</code>
リンクライブラリ	<code>d3d11, dxgi, d3dcompiler, windowscodecs, ole32, user32, gdi32</code>
ビルド手段	① <code>MountainParticles.sln</code> (Release/Debug × x64) ② <code>build.bat</code> (vswhere → vcvars64 → cl 一括)
出力	<code>bin\Release\MountainParticles.exe</code> / <code>bin\MountainParticles.exe</code>
リソース検索	実行時に「カレント → exe → exe¥.. → exe¥..¥..」の順に <code>shaders\Common.hlsl</code> を 探し、同階層の <code>assets</code> も登録

ソースは UTF-8 (日本語コメント) で保存し `/utf-8` を付ける。`build.bat` のみ cmd.exe 用に Shift-JIS + CRLF で保存している (LF のみだと `for /f` の解釈が壊れる) 。

4. 地形生成（手続き的高さマップ）

地形は `assets/terrain.obj` があればそれを読み込み（6 章）、なければ本章の手続き生成を用いる。どちらの場合も最終的に「高さマップ + 描画メッシュ」という同じ形になる。

4.1 高さ関数

地形は $y = h(x, z)$ の高さマップで表す。谷の中心線を正弦波の和で蛇行させ、中心からの距離に応じて山を立ち上げる。上流（ $-z$ ）を高く、下流（ $+z$ ）を低くして水が流れる勾配を作る。

$$c(z) = 4 \sin(0.16 z) + 2 \sin(0.41 z + 1.7) \quad (\text{谷の中心線})$$
$$d = |x - c(z)|, \quad w = \text{smoothstep}(1.6, 8.5, d) \quad (0: \text{谷底}, 1: \text{山})$$
$$h = -0.20 z + 0.3 \cdot \min(d, 1.6)^2 + 11 w + \text{fBm}(x, z) \cdot (0.25 + 5 w) + \text{ridged}(x, z) \cdot 1.5 w$$

項	役割
$-0.20 z$	全体の傾斜（60 m で 12 m の落差、約 11° ）
$0.3 \cdot \min(d, 1.6)^2$	谷底の浅い U 字断面。水を中央へ集める
$11 w$	両側の山（smoothstep で滑らかに立ち上げ）
$\text{fBm} \times (0.25 + 5 w)$	岩肌の凹凸。谷底は ± 0.25 m、山肌は ± 5 m と場所で振幅を変える
$\text{ridged} \times 1.5 w$	稜線状の鋭い筋（尾根ノイズ）

4.2 ノイズ

Ken Perlin の Improved Noise（2002）を 2 次元化して実装。格子点にハッシュで勾配ベクトルを割り当て、評価点との内積を緩和曲線 $6t^5 - 15t^4 + 10t^3$ で双線形補間する。

- **fBm**: 周波数 2 倍・振幅 1/2 のノイズを 6 層重ね、振幅合計で正規化（ $-1 \sim 1$ ）。
- **尾根状ノイズ**: $(1 - |\text{noise}|)^2$ を重ねる。ノイズが 0 を横切る場所に鋭い稜線が出る。
- **周期版**: 格子座標を周期で折り返し、タイリング可能なテクスチャを作る（5 章）。

4.3 メッシュ・法線・UV

- 257×257 頂点、 256×256 セル $\times 2$ 三角形 = 131,072 三角形、32bit インデックス。
- 法線は中心差分 $n = \text{normalize}(-\partial h / \partial x, 1, -\partial h / \partial z)$ 。SPH の衝突にも同じ関数を使う。
- $UV = (x, z) \times 0.12$ （約 8.3 m でテクスチャ 1 周、WRAP サンプリング）。
- 三角形の巻き順は D3D 既定（画面上で時計回りが表面、左手系）に合わせ、上から見て時計回りに並べる。
- 任意座標の高さは双線形補間 `GetHeight(x, z)` で返す（読み取り専用でスレッドセーフ）。

5. 岩テクスチャ（WIC 読み込みと手続き生成）

5.1 画像ファイルの読み込み（TextureLoader）

Windows Imaging Component（WIC）を用い、PNG / JPG / BMP / TIFF / GIFなどを外部ライブラリなしで読み込む。

1. `CoInitializeEx` → `IWICImagingFactory` を静的に 1 回生成
2. `CreateDecoderFromFilename` でファイルを開き、先頭フレームを取得（形式は拡張子でなく内容で判定）
3. `IWICFormatConverter` で 32bpp RGBA に統一（パレット・グレースケール・CMYK も吸収）
4. `CopyPixels` でメモリへ展開（行ピッチ = 幅 × 4）
5. `MipLevels = 0 + D3D11_RESOURCE_MISC_GENERATE_MIPS` でテクスチャを作成し、レベル 0 に `UpdateSubresource`、`GenerateMips` で縮小版を自動生成

`assets/rock.png` → `rock.jpg` → `rock.jpeg` → `rock.bmp` の順に探し、すべて失敗した場合は 5.2 の手続き生成に切り替える。タイトルバーに出所（画像 / ノイズ生成）を表示する。

5.2 手続き生成（フォールバック）

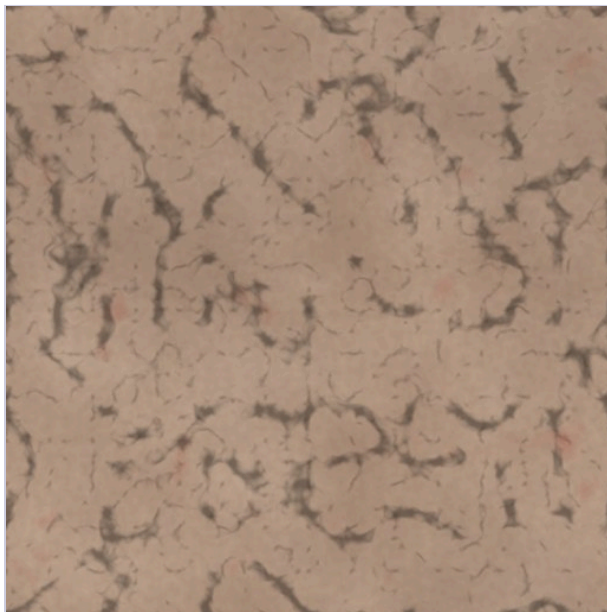


図 5-1 同梱の rock.png（1024×1024、シームレス）

1024×1024 の各ピクセルで周期ノイズを評価し、以下を合成する。周期ノイズを使うことで端と端が連続し、WRAP で敷き詰めても継ぎ目が出ない。

成分	ノイズ	効果
まだら	fBm（周期 4, 6 層）	明るい肌色と茶色の混合比
赤斑	fBm（周期 9, 別位相）	赤みのある斑点
亀裂	尾根ノイズ（周期 10）を smoothstep(0.76, 0.93) で閾値処理	細い暗い線
大きな明暗	fBm（周期 2, 3 層）	広い範囲のムラ
ざらつき	fBm（周期 64）	微細な粒状感

5.3 サンプリングとシェーディング（Terrain.hlsl）

- サンプラー: 異方性フィルタ 8 × WRAP。斜めから見る谷の壁でもぼけにくい。
- 繰り返し感の低減: 別スケール（×0.23、オフセット 0.31）でもう一度サンプルして乗算する。
- 陰影: ランバート拡散反射 $\max(0, N \cdot L)$ + 半球環境光（上向きほど明るい）+ 谷底の暗色化。光源方向は (−0.35, 0.85, 0.45) を正規化。

6. OBJ 地形の取り込み（Blender 連携）

6.1 目的と方式

手続き生成の地形に加え、Blender などの DCC ツールで作ったメッシュを地形として使えるようにした。方式は「メッシュはそのまま描画し、衝突判定用には真上から高さマップに焼き込む」というハイブリッドである。SPH 側は既存の `GetHeight` / `GetNormal` をそのまま使うため、物理コードの変更なしに任意形状の地形を扱える。

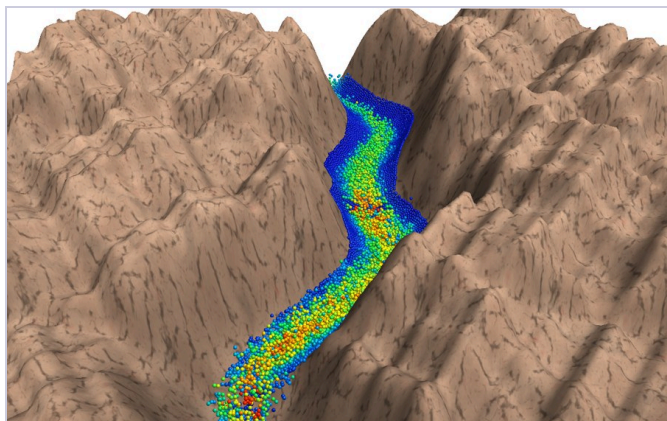


図 6-1 OBJ 地形（試作用の谷、2 段の滝つき） — 粒子モード



図 6-2 同 — 表面生成モード

6.2 OBJ ロダー（MeshLoader）

Wavefront OBJ は行指向のテキスト形式で、外部ライブラリなしで読める。対応要素と処理は次のとおり。

行	内容	処理
<code>v x y z</code>	頂点座標	一時配列に蓄積
<code>vt u v</code>	テクスチャ座標	$v \leftarrow 1 - v$ （OBJ は左下原点、D3D は左上原点）
<code>vn x y z</code>	法線	蓄積。無ければ後で面積重み付きスムーズ法線を計算
<code>f a/b/c ...</code>	面	<code>a</code> / <code>a/b</code> / <code>a//c</code> / <code>a/b/c</code> と負の相対インデックスに対応。(v, vt, vn) の組をキーに頂点を共有し、4 頂点以上は扇状 (0, i, i+1) に三角形化
<code>o g s usemtl mtl1lib</code>	その他	無視（材質は本プログラム側の岩テクスチャを使う）

OBJ では「座標が同じでも UV や法線が異なれば別頂点」なので、(v, vt, vn) の三つ組をハッシュマップのキーにして頂点を生成・再利用する。試作用の谷（121×121 頂点、14,400 四角形）は約 2 MB で、読み込みは数十 ms。

6.3 座標系の変換

Blender は右手系・Z-up、DirectX は左手系・Y-up である。変換は 2 段階で行う。

1. **エクスポート時**（Blender 側）：「Forward: -Z, Up: Y」を指定すると Blender $(x, y, z) \rightarrow$ OBJ $(x, z, -y)$ と軸が入れ替わり、右手系 Y-up になる。

2. 読み込み時（本プログラム）：右手系 → 左手系にするため x を反転する（位置と法線）。

Blender $(x, y, z) \rightarrow \text{OBJ} (x, z, -y) \rightarrow \text{DirectX} (-x, z, -y)$

結果として Blender の上面図で「上 (+Y)」が奥（上流、 $-z$ ）、「右 (+X)」が画面右になり、Blender で見たままの構図でカメラに映る。

巻き順の統一：鏡映変換で三角形の巻き順が反転し、さらに元データの巻き順は保証されない。そこで三角形ごとに $\text{cross}(\mathbf{e}_1, \mathbf{e}_2) \cdot \mathbf{y}$ を計算し、負（下向き）なら頂点 1, 2 を入れ替える。地形は「上から見える面が表」なので、4 章のメッシュと同じ規約（左手系 + 時計回りが表 $\Leftrightarrow \text{cross} \cdot \mathbf{y} > 0$ ）に揃う。

6.4 自動フィット

SPH のパラメータ（粒子間隔 0.3 m、地形 60 m 四方）に合わせるため、読み込んだメッシュのバウンディングボックスから一様スケールを決める。

$s = 0.999 \cdot \text{WORLD_SIZE} / \max(\Delta x, \Delta z)$, $\mathbf{p} \leftarrow (\mathbf{p} - \text{center}) \cdot s$

xz の大きい方の辺が 60 m になり、中心が原点に来る。高さも同じ倍率で拡張されるので、Blender 側では縦横比だけ意識すればよい。0.999 は端の頂点が高さマップ格子の外に出ないための余裕である。

6.5 高さマップへの焼き込み（BakeHeightMap）

衝突判定は高さマップ（257×257）で行うため、メッシュを真上から「ラスタライズ」して各格子点の高さを求める。

```
heights[*] = 未設定
for 各三角形 (p0, p1, p2):
    xz 平面に投影し、包含する格子範囲 [ix0, ix1] × [iz0, iz1] を求める
    for 範囲内の各格子点 (x, z):
        辺関数で重心座標 (w0, w1, w2) を計算
        if w0, w1, w2 ≥ -ε:                                # 三角形の内側（辺上を含む）
            y = w0·p0.y + w1·p1.y + w2·p2.y
            heights[x, z] = max(heights[x, z], y)          # 上から見える面を採用
for 未設定の格子点: 覆われた 4 近傍の平均で埋める（全て埋まるまで反復）
```

- 重心座標は辺関数（2D の外積）を面積で割って求める。面積 ≈ 0 の縮退三角形（真横を向いた面）はスキップ。
- max を取るため、オーバーハングがあっても上面のみが残る（下面は衝突に使われない）。
- メッシュが正方形でない場合や穴がある場合、未被覆の格子点は隣接平均で外挿される（平坦に延長）。
- 計算量は $O(\text{三角形数} \times \text{各三角形の包含格子点数})$ 。14,400 四角形で数 ms。

6.6 エミッタ位置の決定

手続き生成では谷の中心線 $c(z)$ を式で持っていたが、任意メッシュでは使えない。`Terrain::GetValleyCenterX(z)` を導入し、OBJ 地形では「その z の行で高さが最も低い x 」を高さマップから線形探索する（端 1 m は除外）。エミッタは上流端 $z = -27.5$ でこの関数を呼ぶため、谷がどこにあっても水はその谷底に投入される。

6.7 Blender 側の手順と同梱スクリプト

手順	内容
1. モデリング	上面図で上 (+Y) が上流 = 奥、下 (-Y) が下流 = 手前になるように谷を作る。1 単位 = 1 m にすると直感的だが、自動フィットされるので厳密でなくてよい
2. UV	任意。無ければワールド座標から $(x, z) \times 0.12$ で自動生成
3. エクスポート	File → Export → Wavefront (.obj)。Forward: -Z, Up: Y。材質は不要
4. 配置	<code>assets/terrain.obj</code> に保存して起動。タイトルバーに「地形: OBJ」

- `tools/blender_export_valley.py`: Blender 内で実行（または `blender -b -P`）すると、bmesh で 121×121 の格子を生成し高さ関数で変位、UV を設定して上記設定で OBJ を書き出す。Blender 2.8~3.2（`export_scene.obj`）と 3.3 以降（`wm.obj_export`）の両 API に対応。
- `tools/make_test_valley.py`: Blender が無い環境向けに、同じ高さ関数・同じ軸変換で OBJ を書き出す。同梱の `assets/terrain.obj` はこれで生成した。

試作用の谷は手続き生成版と区別できるよう、蛇行の周期を変え、途中に 2 段の落差（高さ 2 m の滝）を入れている。図 6-1 の上流側で水が青く溜まっているのは、この段差の上に淵ができているためである。

6.8 制約

- 高さマップ方式のため、洞窟やひさし状のオーバーハングの下面には衝突しない。必要なら符号付き距離場（SDF）を構築し、SPH の衝突処理を「最近傍面への押し戻し」に置き換える（16 章）。
- 高さマップ解像度は 257×257 （約 0.23 m 刻み）。これより細かい段差は平滑化される。
- 地形シェーダーの谷底暗色化（ $y + 0.2z$ による）は手続き生成の傾斜を前提にしており、OBJ 地形では見た目上の近似になる。

7. SPH 流体シミュレーション

7.1 SPH の基本式

SPH (Smoothed Particle Hydrodynamics) は流体を粒子で離散化し、任意の物理量 A を近傍粒子の値をカーネル W で重み付けした和で近似する。

$$A(x) \approx \sum_j m_j (A_j / \rho_j) W(|x - x_j|, h)$$

本実装は Müller et al. (2003) の WCSPH (弱圧縮性 SPH) に従う。

$$\text{密度: } \rho_i = \sum_j m W_{\text{poly6}}(r_{ij}, h)$$

$$\text{圧力 (状態方程式): } p_i = k \cdot \max(\rho_i - \rho_0, 0)$$

$$\text{圧力力: } \mathbf{f}_i^p = -\sum_{j \neq i} m (p_i + p_j) / (2\rho_j) \nabla W_{\text{spiky}}(r_{ij}, h)$$

$$\text{粘性力: } \mathbf{f}_i^v = \mu \sum_{j \neq i} m (\mathbf{v}_j - \mathbf{v}_i) / \rho_j \nabla^2 W_{\text{visc}}(r_{ij}, h)$$

$$\text{加速度: } \mathbf{a}_i = (\mathbf{f}_i^p + \mathbf{f}_i^v) / \rho_i + \mathbf{g}$$

圧力を $\max(\cdot, 0)$ で打ち切るのは、負圧 (引力) による粒子の凝集を防ぐためである (自由表面付近では密度が ρ_0 を下回るため)。

7.2 カーネル関数

$$W_{\text{poly6}}(r) = 315 / (64\pi h^9) \cdot (h^2 - r^2)^3 \quad (0 \leq r \leq h)$$

$$\nabla W_{\text{spiky}}(r) = -45 / (\pi h^6) \cdot (h - r)^2 \cdot \hat{\mathbf{r}} \quad (0 < r \leq h)$$

$$\nabla^2 W_{\text{visc}}(r) = 45 / (\pi h^6) \cdot (h - r) \quad (0 \leq r \leq h)$$

- poly6 は平方根不要で密度計算が軽い。
- spiky は $r \rightarrow 0$ で勾配が消えないため、粒子が重なったときにも斥力が働く (poly6 の勾配だと 0 になり粒子が凝集する)。
- visc のラプラシアンは常に正で、速度差を必ず減らす方向に働く (数値的に安定)。
- 係数は h から起動時に事前計算する。

7.3 パラメータの設計

量	値	根拠
粒子間隔 s	0.30 m	谷底幅 ≈ 3.2 m に約 10 個並ぶ粒度
平滑化半径 h	0.60 m = $2s$	静止状態で約 33 個の近傍を持つ
質量 m	$\rho_0 s^3 = 27$ kg	粒子 1 個が占める体積 $\times$ 水の密度
剛性 k	2000	音速 $c = \sqrt{k} \approx 45$ m/s。深さ 1 m で圧縮率 0.5% ($\rho g H / k$)
粘性 μ	0.40	実際の水より大きい、流れをまとめ数値振動を抑える
時間刻み Δt	0.003 s	CFL 条件 $\Delta t < 0.4 h / (c + v_{\max}) \approx 0.0044$ s を満たす

7.4 時間積分

シンプレクティック・オイラー法（速度を先に更新してから位置を更新）を用いる。エネルギーが長期的にドリフトしにくく、SPH の標準的な選択である。

$$\mathbf{v}_i \leftarrow \mathbf{v}_i + \mathbf{a}_i \Delta t, \quad \mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i \Delta t$$

1 フレームで `ceil(frameTime /  $\Delta t$ )` 回のサブステップを実行するが、上限 `maxSubsteps = 5` で頭打ちにする。処理落ち時にサブステップが増えてさらに遅くなる「死のスパイラル」を防ぎ、その場合は実時間より遅く進行する。

7.5 1 サブステップの処理順

<code>Emit(dt)</code>	エミッタ: <code>spacing/emitSpeed</code> 秒ごとに 14×4 の粒子板を上流に置く
<code>BuildGrid()</code>	空間ハッシュ格子の構築 (8 章)
<code>BuildNeighborLists()</code>	近傍リスト作成 (27 セル走査、最大 96 個)
<code>ComputeDensityPressure()</code>	密度・圧力
<code>ComputeForces()</code>	圧力力 + 粘性力 $\rightarrow$ 加速度クランプ $\rightarrow$ 重力
<code>Integrate()</code>	速度・位置更新 $\rightarrow$ <code>ResolveCollision</code> (9 章)
<code>RemoveDrained()</code>	下流端を越えた粒子を末尾と入れ替えて削除

|| は OpenMP `parallel for` (`schedule(dynamic, 64)`) で並列化した処理。粒子ごとに独立して書き込むため競合がない。

7.6 エミッタと排水

- 上流端 $z = -27.5$ の谷の中心に、粒子板（横 $14 \times$ 縦 4 ）を初速 4 m/s (+z) で投入。板同士が重ならないよう「板が 1 粒子間隔進む時間 = $s / v_{\text{emit}} = 0.075$ s」ごとに投入する（約 750 粒子/秒）。
- 格子配置の規則性を崩すため各粒子に $\pm 15\%$ のジッタ（xorshift 乱数）を加える。
- 下流端 $z > 29.5$ に達した粒子は配列末尾と入れ替えて削除（ $O(1)$ ）。最大 12,000 個で定常流になる。

8. 近傍探索（空間ハッシュ + カウンティングソート）

SPH の計算量は近傍探索に支配される。全粒子対を調べると $O(N^2) = 1.4$ 億対 / ステップとなり実用にならないため、一辺 h の格子で空間を分割し、周囲 27 セルのみを調べる。

8.1 空間ハッシュ

セル座標: $(c_x, c_y, c_z) = \lfloor \mathbf{x} / h \rfloor$

ハッシュ: $\text{key} = (c_x \cdot 73856093) \oplus (c_y \cdot 19349663) \oplus (c_z \cdot 83492791) \& (\text{tableSize} - 1)$

Teschner et al. (2003) の大きな素数によるハッシュ。無限に広い空間を固定サイズ（粒子数の 2 倍以上の 2 のべき乗 = 32768）の表に写像するため、地形全体を格子で埋める必要がない。剰余の代わりにビットマスクを使う。

8.2 カウンティングソート

1. 各粒子のセル座標とハッシュ値を計算 (並列)
2. `cellStart[key + 1]++` でハッシュ値ごとの粒子数を数える
3. 累積和 `cellStart[k+1] += cellStart[k]` → 各ハッシュ値の開始位置
4. `sorted[cellStart[key] + fill[key]++] = i` で粒子番号を振り分け

結果、ハッシュ値 `key` のセルに属する粒子は `sorted[cellStart[key] .. cellStart[key+1])` で連続して取り出せる。動的メモリ確保がなく、毎ステップ $O(N + \text{tableSize})$ で再構築できる。

8.3 近傍リストのキャッシュとハッシュ衝突対策

- 密度計算と力計算は同じ近傍を使うため、走査を 1 回にまとめて粒子ごとに最大 96 個の近傍番号を記録する (`BuildNeighborLists`)。走査コストが半減し、実測で約 3 倍の高速化。
- 異なるセルが同じハッシュ値を持つと、27 セル走査で同じ粒子を 2 回数える危険がある。候補粒子のセル座標が「今調べているセル座標」と一致するかを確認して除外する。
- 距離判定は $r^2 < h^2$ で行い平方根を避ける。

9. 境界処理と安定化

9.1 地形との衝突

粒子中心が「地面の高さ + 粒子半径」より低ければ、その高さまで押し戻し、地面法線 $\mathbf{n}$ に対してめり込む速度成分を除去する。

$$y \leftarrow h(x, z) + s/2, \quad \mathbf{v} \leftarrow \mathbf{v} - (1 + e)(\mathbf{v} \cdot \mathbf{n})\mathbf{n} \quad (\mathbf{v} \cdot \mathbf{n} < 0 \text{ のとき}), \quad \mathbf{v} \leftarrow (1 - \mu_f)\mathbf{v}$$

$e = 0$ （反発なし）、 $\mu_f = 0.01/\text{ステップ}$ 。側壁（ $x = \pm 29.5$ ）と上流壁（ $z = -29.5$ ）も同様に処理する。

9.2 谷からの飛び出しを防ぐ 3 つの安定化

初期実装では壁際で密度が急上昇したときの圧力の「蹴り」で粒子が谷の外へ飛散した。以下の 3 手法で解消した。

手法	内容	パラメータ
加速度クランプ	圧力・粘性による加速度の大きさを上限で打ち切る（重力は別に加算）。1 ステップの速度変化を 0.75 m/s 以下に制限	<code>maxAcceleration = 250</code>
接触ゾーン減衰	地面から 0.3 m 以内の粒子を、地面に近いほど強く減衰（最下部で 6%/ステップ、上端で 0 に線形減少）。壁に当たった粒子の跳ね上がりを抑える	<code>contactZone = 0.30,</code> <code>contactDamping = 0.06</code>
孤立粒子の空気抵抗	近傍が 8 個未満の粒子（飛沫）に 1.5%/ステップの減衰。勢いを失いすぐ流れに戻る	<code>airDrag = 0.015,</code> <code>sparseNeighbors = 8</code>

このほか安全弁として速度上限 10 m/s、密度 0 除算防止（ $\rho^2 < 10^{-12}$ の粒子対を除外）を設けている。

9.3 境界密度の欠損について

地面の下には粒子が存在しないため、接地粒子の密度は本来より低く見積もられる（境界密度欠損）。本実装は境界粒子を置かず、衝突処理と接触ゾーン減衰で吸収している。より厳密には Akinci らの境界粒子法や ghost particle が有効である（16 章）。

10. 粒子のレンダリング（スフィア・インポスター）

10.1 ビルボード生成

粒子ごとに 4 頂点の四角形を頂点バッファなしで生成する。`DrawInstanced(4, N)` を発行し、頂点シェーダーで `SV_InstanceID` から粒子を、`SV_VertexID` から四隅 $(-1, -1)$ $(-1, 1)$ $(1, -1)$ $(1, 1)$ を選ぶ。四角形はビュー空間で作るため常にカメラに正対する。

```
float4 p = Particles[iid]; // StructuredBuffer<float4> (xyz: 位置, w: 速さ)
float3 centerV = mul(float4(p.xyz, 1), View).xyz;
float3 posV = centerV + float3(corners[vid] * radius, 0);
o.posH = mul(float4(posV, 1), Proj);
```

10.2 球面の再構成と正しい深度

ピクセルシェーダーで四角形内座標 uv $(-1 \sim 1)$ から球面上の点を求める。左手系ビュー空間ではカメラは $+z$ を向くので、カメラ側の半球は $n_z < 0$ 。

$$r^2 = u^2 + v^2 > 1 \rightarrow \text{discard}$$
$$\mathbf{n} = (u, v, -\sqrt{1 - r^2}), \quad \mathbf{p}_V = \mathbf{c}_V + \mathbf{n} R$$
$$\text{depth} = (\mathbf{p}_V \cdot \text{Proj}).z / (\mathbf{p}_V \cdot \text{Proj}).w \rightarrow \text{SV_Depth}$$

`SV_Depth` に球面の深度を書くことで、平面の四角形でありながら球同士・地形と正しく交差する。早期 Z は無効になるが、粒子数 12,000 程度では問題にならない。

10.3 速度による着色

- 参考動画に合わせ、速さ $0 \sim 6$ m/s を「濃青 → 水色 → 緑 → 黄 → 赤」の 5 色制御点の区間線形補間（Jet カラーマップ）で表す。
- SPH の速度は粒子ごとに細かく揺らぐため、そのまま着色すると色がまだらになる。CPU 側で近傍リストを使い「自分と近傍の速さの平均」を描画用の速さとして渡す（`GetRenderData`）。物理には影響しない。
- 陰影は拡散反射 + Blinn-Phong 鏡面反射（指数 40）。

11. 表面生成（スクリーンスペース流体レンダリング）

粒子群から陽なメッシュ（Marching Cubes 等）を作る代わりに、画面空間で深度を平滑化して法線を復元する手法（van der Laan, Green, Sainz 2009）を簡略化して実装した。3 パスで構成される。

11.1 パス 1: 深度

粒子を 10 章と同じ球インポスターで描き、色の代わりにビュー空間の z を `R32_FLOAT` テクスチャに出力する。メインの深度バッファ（地形描画済み）と組で描くため、地形の後ろの粒子は自動的に隠れる。テクスチャは 0 でクリアし、0 = 流体なしとして扱う。表面用の球半径は粒子間隔の 1.3 倍とし、隣の球と重なって隙間を作らない。

11.2 パス 2: バイラテラルぼかし（分離可能）

$$w_k = \exp(-k^2 / 2\sigma_s^2) \cdot \exp(-(d_k - d_0)^2 / 2\sigma_d^2), \quad d = \sum w_k d_k / \sum w_k$$

- **半径の透視補正:** ワールド半径 0.9 m を画面に投影したピクセル数 $\text{radiusPx} = 0.9 / d_0 \cdot \text{Proj}_{11} \cdot H/2$ を使う（2～24 px）。近い流体ほど広く、遠い流体ほど狭くぼかすため見た目の滑らかさが距離に依らない。
- **流体外の除外:** $d \leq 0$ （流体なし）のサンプルは重みに入れない。これにより輪郭が内側へ縮まず、地形との境界がぼけない。
- **深度 $\sigma = 1.5$ m と大きめ。** 流体外は上記で除外済みなので、球の凹凸（ ≤ 0.4 m）を確実に平滑化できる。初期値 0.35 では球が個別に残り「ブロック状」の見た目になった。
- 横 → 縦の分離可能フィルタを 4 回反復（計 8 パス）。1 パスあたり最大 49 タップ。

11.3 パス 3: 法線復元と合成

ぼかした深度からビュー空間座標を復元し、隣接ピクセルとの差分から法線を求める。

$$\begin{aligned} p(u, v, d) &= (\text{ndc}_x \cdot d / \text{Proj}_{00}, \text{ndc}_y \cdot d / \text{Proj}_{11}, d) \\ \partial p / \partial x &= \text{左右どちらか } |\Delta z| \text{ の小さい側}, \quad \partial p / \partial y = \text{上下どちらか } |\Delta z| \text{ の小さい側} \\ \mathbf{n} &= \text{normalize}(\partial p / \partial x \times \partial p / \partial y) \end{aligned}$$

差の小さい側を採用するのは、流体の輪郭（深度が飛ぶ場所）で法線が壊れるのを避けるためである。ピクセルの $+y$ は下向き = ビュー空間の $-y$ なので、外積 $(+x) \times (-y) = -z$ がカメラ側を向く法線になる。

陰影は水らしさを出すため 3 成分を合成し、 $\alpha = 0.9$ のアルファブレンドで地形の上に描く。

$$\begin{aligned} \text{拡散: } C_w(0.5 + 0.5 \max(0, \mathbf{n} \cdot \mathbf{L})), \quad \text{鏡面: } 0.35 \cdot \max(0, \mathbf{n} \cdot \mathbf{H})^{48} \\ \text{フレネル (Schlick): } F = 0.06 + 0.94 (1 - \mathbf{n} \cdot \mathbf{V})^4 \rightarrow \text{空色を } F \cdot 0.5 \text{ で混合} \end{aligned}$$

$C_w = (0.64, 0.68, 0.78)$ は参考動画の灰青色に合わせた。

12. 描画パイプラインと定数バッファ

12.1 定数バッファ（b0）

全シェーダーで 1 つの `cbuffer CFrame`（320 バイト）を共有する。C++ 側 `FrameConstants` と配置を一致させ、`static_assert` で 16 バイト境界を検証している。

メンバ	内容
View / Proj / ViewProj	行列（転置済み）
CameraPosW, LightDirW, LightDirV	カメラ位置、光源方向（ワールド / ビュー）
ScreenSize	幅, 高さ, 1/幅, 1/高さ
Params	x: 粒子半径, y: 時間, zw: ぼかし方向

行列の転置について: DirectXMath は行優先で $v' = v M$ の規約、HLSL の既定は列優先パッキング。CPU 側で転置して転送すると、HLSL で `mul(float4(v,1), M)` と書け、`M._m00`, `M._m11` も数学どおりの要素になる（法線復元で使用）。

12.2 ステートとリソース

種別	用途
ラスタライザ（裏面カリング / なし）	地形 / ビルボード・フルスクリーン三角形
深度（有効 / 無効）	地形・粒子 / ぼかし・合成
ブレンド（不透明 / アルファ）	通常 / 水面の合成
サンプラー（異方性 WRAP / ポイント CLAMP）	岩テクスチャ / 深度テクスチャ
StructuredBuffer（DYNAMIC）	粒子 12,000 × float4。毎フレーム Map(WRITE_DISCARD)
R32_FLOAT × 2（RTV + SRV）	深度のピンポンぼかし。リサイズ時に再作成

フルスクリーンパスは `SV_VertexID` から $(-1,1)(3,1)(-1,-3)$ の 1 枚の三角形を生成し、画面全体を覆う（四角形 2 枚より対角線上の重複処理がない）。

12.3 リサイズ処理

`WM_SIZE` で RTV/DSV を解放 → `ResizeBuffers` → 再作成。続けて `FluidRenderer` の深度テクスチャも作り直す。初期化完了前の `WM_SIZE` はフラグで無視する。

13. カメラ・入力・メインループ

13.1 オービットカメラ

$$\text{pos} = \text{target} + D(\cos\varphi \sin\theta, \sin\varphi, \cos\varphi \cos\theta)$$

θ (yaw) , φ (pitch, $\pm 89^\circ$ にクランプ) , D (3~300 m) 。既定は下流側 +z から上流を見下ろす構図 (pitch 34° , D 42) で、参考動画と同じく水が奥から手前へ流れて見える。パンはカメラの右・上ベクトル方向に注視点を動かす。

13.2 メインループ

```
while (msg != WM_QUIT) {
    if (PeekMessage) { Dispatch; continue; }    // 入力を全て処理してから描画
    dt = steady_clock 差分
    if (!paused) sph.Update(dt)                // 物理 (サブステップ上限あり)
    gfx.BeginFrame(白)
    UpdateFrameConstants()                     // View/Proj/光源 → b0
    terrain.Render()
    fluidRenderer.Render(sph.GetRenderData(), mode)
    gfx.EndFrame(vsync)
    FPS 計測 / タイトル更新
}
```

14. 性能評価

段階	粒子数	FPS	備考
初期実装（毎パス近傍走査、サブステップ可変）	8,300	20～24	処理落ち → サブステップ増加のスパイラル
近傍リストキャッシュ + 2 のべき乗ハッシュ + サブステップ上限	10,500	106～116	約 5 倍
安定化（接触ゾーン等）追加後、粒子モード	12,000	63～120	密度が上がり近傍数増
同、表面生成モード	12,000	54～80	ばかし 8 パス分の GPU 負荷

測定環境: 8 論理コア CPU、1280×800、vsync 有効（上限 120 fps 表示はモニタ依存）。

14.1 計算量の内訳（1 サブステップ、12,000 粒子）

- 近傍候補: 27 セル × 平均 8 粒子 ≈ 216 候補/粒子 → 2.6 M 距離判定
- 近傍数: 約 33～60/粒子 → 密度・力の合計で約 1.1 M カーネル評価
- 格子構築: $O(N + 32768)$ 、逐次部分は約 0.1 ms
- GPU 転送: 192 KB / フレーム（PCIe 帯域に対して無視できる）

15. 調整パラメーター一覧（SPHParams）

パラメータ	既定値	意味・調整の目安
particleSpacing	0.30 m	小さくすると細かいが粒子数が 3 乗で増える
smoothingRadius	0.60 m	間隔の 2 倍を維持する
restDensity	1000	水
stiffness	2000	上げると非圧縮に近づくが Δt を小さくする必要がある
viscosity	0.40	大きいほど粘る。安定条件の目安 < 1.0
gravity	9.8	
maxSpeed / maxAcceleration	10 / 250	安全弁。飛散が出る場合は下げる
timeStep / maxSubsteps	0.003 / 5	1 フレーム最大 15 ms ぶん進む
restitution / friction	0 / 0.010	反発 / 接地摩擦
contactZone / contactDamping	0.30 / 0.06	飛び出し抑制。流れを速くするなら damping を 0.03～0.04 に
airDrag / sparseNeighbors	0.015 / 8	飛沫の減衰
maxParticles / maxNeighbors	12000 / 96	メモリ: 近傍リスト 12000×96×4 B = 4.6 MB
emitWidth / emitHeight / emitSpeed	14 / 4 / 4.0	投入量 $\approx 56 \times \text{emitSpeed} / \text{spacing}$ 粒子/秒
emitZ / emitLift	-27.5 / 0.8	投入位置

描画側: MAX_SPEED_FOR_COLOR（6 m/s、Particle.hsl）、BLUR_WORLD_RADIUS / BLUR_DEPTH_SIGMA（0.9 / 1.5、FluidSurface.hsl）、ぼかし反復回数（4、FluidRenderer.h）。

16. 既知の制約と拡張案

制約	拡張案
CPU SPH のため粒子数は 1〜2 万が上限	コンピュータシェーダーへ移植（格子構築は GPU 基数ソート、密度・力はスレッドグループ単位）。10 万粒子以上が可能
WCSPH は圧縮率が k に依存し、 k を上げると Δt が小さくなる	PCISPH / IISPH / DFSPH など圧力を反復で解く非圧縮手法
境界密度の欠損を減衰で吸収している	Akinci et al. (2012) の境界粒子（地形表面に粒子をサンプリングして密度に寄与させる）
水面は不透明に近い $\alpha = 0.9$ で厚みを考慮していない	厚みパス（加算ブレンドで粒子の厚さを蓄積）による透過率・屈折の計算
スクリーンスペース法は視点依存で、画面外の情報がない	Marching Cubes / 異方性カーネル (Yu & Turk 2013) による陽なメッシュ生成
表面張力なし	Akinci らの凝集力モデル、または負圧の一部を許容
地形は高さマップ（OBJ 取り込み時もオーバーハング下面は不可）	符号付き距離場（SDF）による任意形状との衝突

17. 付録

17.1 ファイル一覧

パス	行数	内容
src/main.cpp	≈ 340	エントリーポイント、入力、メインループ
src/Graphics.h/.cpp	≈ 420	D3D11 基盤
src/Camera.h/.cpp	≈ 190	オービットカメラ
src/Noise.h/.cpp	≈ 340	ノイズ
src/Terrain.h/.cpp	≈ 800	地形（OBJ 取り込み・焼き込みを含む）
src/TextureLoader.h/.cpp	≈ 200	WIC 画像読み込み
src/MeshLoader.h/.cpp	≈ 280	OBJ 読み込み
src/SPH.h/.cpp	≈ 780	流体
src/FluidRenderer.h/.cpp	≈ 420	粒子・表面描画
shaders/*.hlsl(i)	≈ 450	Common / Terrain / Particle / FluidSurface
docs/API.md, docs/FLOW.md	—	API 仕様書、Mermaid フロー図
assets/rock.png	—	岩テクスチャ（差し替え可）
assets/terrain.obj	—	地形メッシュ（差し替え可、無ければ手続き生成）
tools/blender_export_valley.py	—	Blender 用: 試作用の谷を生成して OBJ 出力
tools/make_test_valley.py	—	Blender なしで同じ OBJ を出力

17.2 参考文献

1. M. Müller, D. Charypar, M. Gross, "Particle-Based Fluid Simulation for Interactive Applications," SCA 2003.
2. W. J. van der Laan, S. Green, M. Sainz, "Screen Space Fluid Rendering with Curvature Flow," I3D 2009.
3. K. Perlin, "Improving Noise," SIGGRAPH 2002.
4. M. Teschner et al., "Optimized Spatial Hashing for Collision Detection of Deformable Objects," VMV 2003.
5. N. Akinci et al., "Versatile Rigid-Fluid Coupling for Incompressible SPH," SIGGRAPH 2012.
6. B. Solenthaler, R. Pajarola, "Predictive-Corrective Incompressible SPH," SIGGRAPH 2009.
7. J. Yu, G. Turk, "Reconstructing Surfaces of Particle-Based Fluids Using Anisotropic Kernels," TOG 2013.
8. C. Schlick, "An Inexpensive BRDF Model for Physically-based Rendering," Eurographics 1994.